World Models · Ray · Robotics · Real-Time Control

Real-time inference for JEPA world model architectures with Ray

With V-JEPA2-AC the system decides what a robot arm should do next by assessing hundreds of possible futures and picking the one closest to the goal. It takes a significant amount of time between moves. This document presents a design for getting that under a second by moving the search into a Ray cluster.

By Omar Shorbaji · September 2026 · 10 min read

01 · A Primer on Robotics

A robot perceives a partial observation of the environment, selects an action via some policy, executes it through a controller, and repeats this at control-loop frequency. Classically each of those three functions is solved in closed form, and a CPU on the robot is enough to run all of them.

Classical robotics built each of those three boxes from first principles: hand-derived kinematics for perception, trajectory optimization for planning, PID or model-predictive controllers for control. This approach, depicted in Figure 1, underlies decades of industrial automation, but every new task, object, or environment typically requires new engineering.

Classical robotics — ~1 ms / action

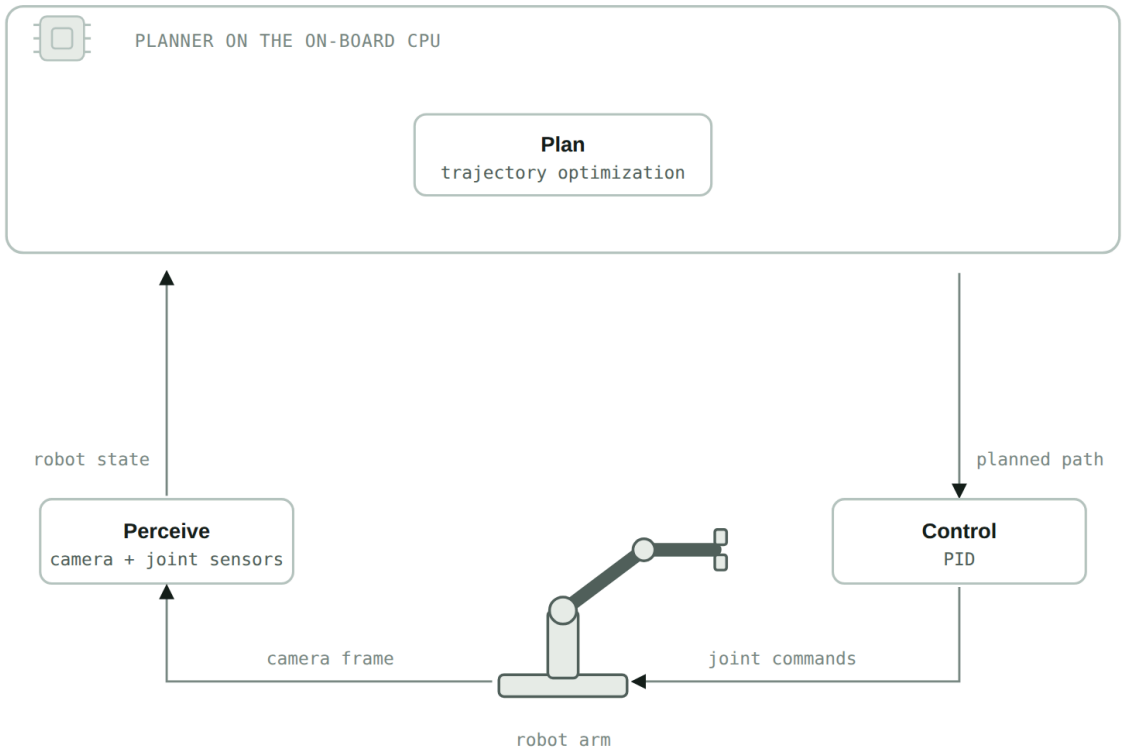


Figure 1. *The loop every robot runs. Perception turns sensor readings into state, planning produces a path, control executes it, and the arm closes the loop through the physical world. All three are solved in closed form on one on-board CPU. What it costs is engineering: the policy only covers the task it was derived for.*

Physical AI aims to replace some or all of those boxes with networks trained on data, trading hand-derived precision for generality: a model trained on sufficient data transfers to situations not explicitly programmed for.

How the JEPA architecture fills the "plan" box

V-JEPA2-AC is one such model. It is built on the *Joint-Embedding Predictive Architecture* (JEPA). In this architecture, the system decides what a robot arm should do next by assessing hundreds of possible futures using the model and picking the one closest to the goal. Figure 2 shows the whole loop in place on the hardware that runs it.

Deployed zero-shot on Franka arms in two labs, with no fine-tuning for the specific environment, V-JEPA2-AC reports 65–80% success on pick-and-place with novel objects. This ran on a single RTX 4090. However, that configuration takes **16 seconds to plan every single action**. It is not fast enough to run continuously.

Where the sixteen seconds actually goes

It is worth walking one planning step end to end, because the arithmetic explains the rest of this document. The arm holds still. The camera frame and the goal image are encoded once. The search algorithm, CEM, then draws 800 candidate action sequences from a Gaussian centered on its current best guess, pushes every one through the predictor, scores the results against the goal embedding, keeps the best 80, and re-centers the Gaussian on them. That is one round, and it happens ten times: roughly 8,000 predictor evaluations for one small movement of one arm. The system then executes only the first action of the winning sequence, throws the rest away, and starts over from a fresh frame. The cost is structural, and it recurs at every step.

V-JEPA2-AC on one on-board GPU — 16 s / action

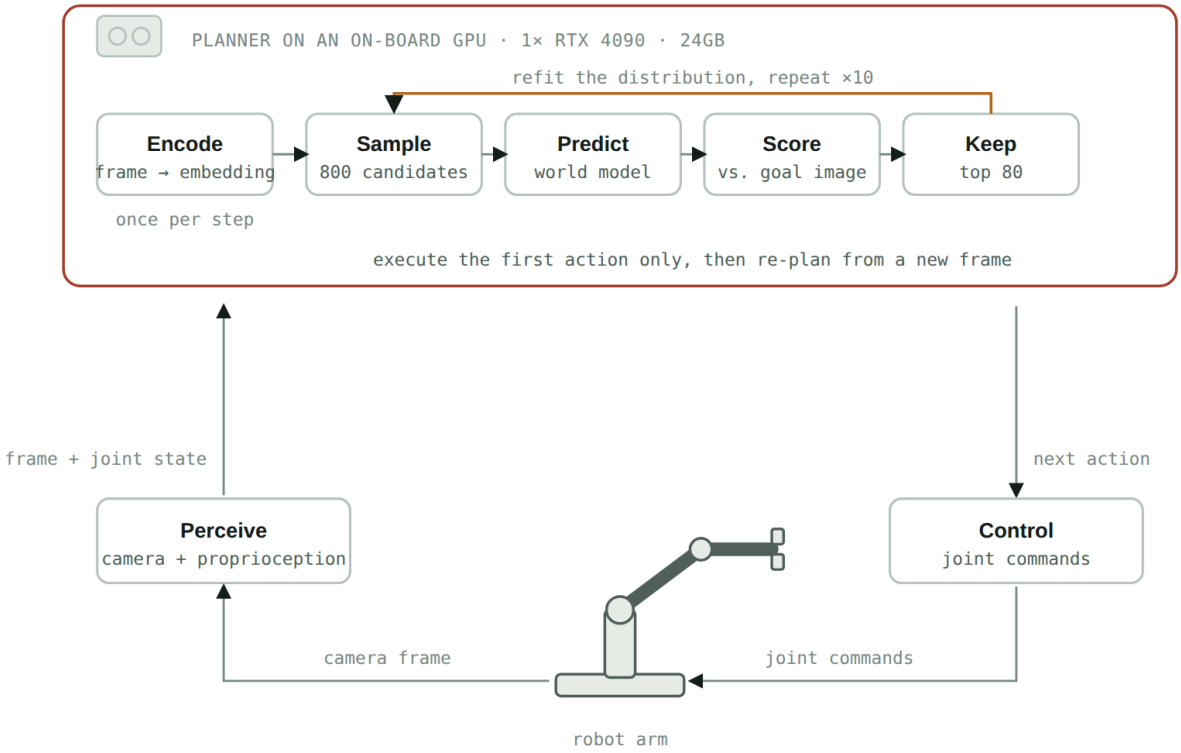


Figure 2. *Only the plan box changes. The robot still captures a camera frame and its joint state, and control is untouched, but the closed-form planner is replaced by the model and a search around it. The encoder is part of the model, so encoding happens here, once per step; the ten-round loop that follows is the expensive part.*

The controller is also **blocking**: it waits for the previous action to complete before sending the next, so planning never overlaps with motion. The robot pauses, plans for sixteen seconds, moves, and pauses again. That stutter, not the model's accuracy, is the problem this document sets out to fix.

02 · The Approach

We take the approach of, first, moving world model inference off the robot and into a cloud server. While this introduces the new cost of a network round trip on every control step, that is offset by other improvements. First, it opens the opportunity of serving the model on a more powerful GPU. In this implementation we use an H100, which has roughly three times the memory of the 4090 used in the paper and significantly more FLOPS.
The same search on one cloud GPU — ≈8.0 s / action

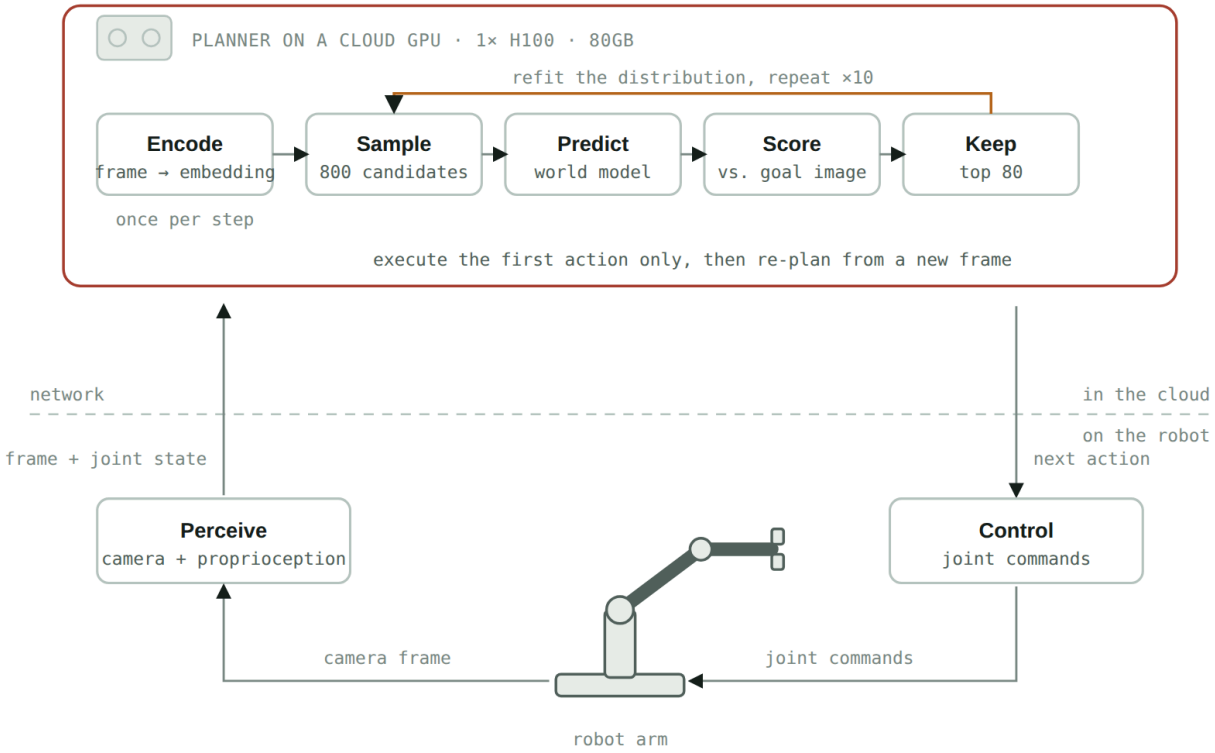


Figure 3. *The plan block lifts the inference off the robot. What stays local is sensing and actuation, so what crosses the network is one camera frame plus its joint state, and one action vector back.*

On its own, this change yields a modeled result of about 8 seconds, down from 16. While twice as fast, the arm still stops between moves.

The second step is to distribute the load across multiple GPUs. The opportunity comes from noting that within a single CEM refinement round rollouts can be computed independently and gathered before the round moves on. The paper runs them as one large batch on one GPU; nothing requires them to stay on one GPU. In our implementation we use four eight-GPU nodes, 32 H100s in total.

The search distributed across a Ray cluster — ≈0.6 s / action

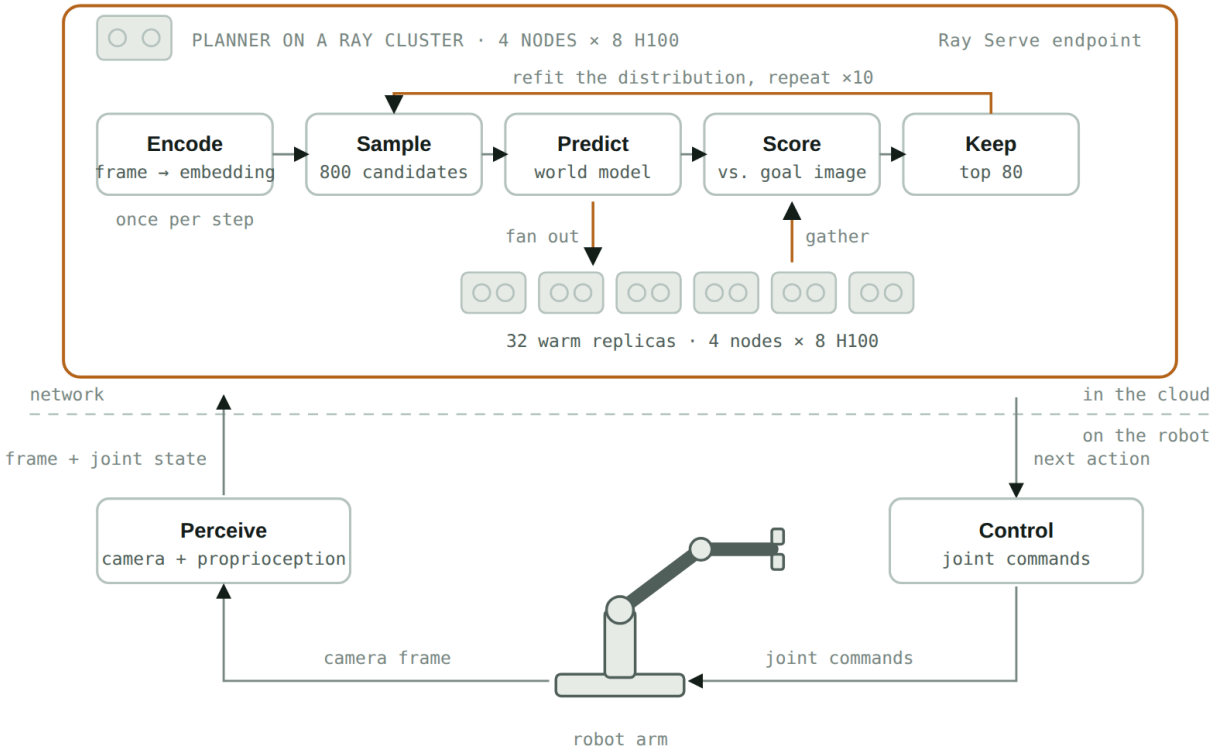


Figure 4. *One step inside the plan block changes. The 800 candidates in a round are independent, so the predict step fans out across warm replicas that already hold the model and gathers before scoring. The ten rounds are still sequential, which is what sets the floor.*
We implement this using Ray.

03 · A Primer on Ray

A general framework for turning Python into a cluster program

Ray is an open-source framework, originally built at UC Berkeley's RISELab and now maintained by Anyscale, for scaling ordinary Python from a laptop to a cluster without rewriting it as a distributed system. It exposes a small set of primitives and preserves mostly-sequential code structure.

Ray Core

Ray Core has two building blocks that matter here. A @ray.remote function becomes a **task**, scheduled somewhere in the cluster and returning a future. A @ray.remote class becomes an **actor**: a stateful worker pinned to a machine, and optionally a GPU, that keeps loaded model weights resident between calls. Tasks suit one-off parallel work; actors suit a process that pays to load a 1.2-billion-parameter checkpoint once and then answers many cheap requests.

Ray Serve

Ray Serve sits on top of Ray Core: a @serve.deployment declaration is enough for Ray to

manage a pool of replicas, route requests, and scale under load. Ray Core answers how to run a function on many workers; Ray Serve answers how to expose it as a service other systems call, which is the shape a robot's control loop needs, a low-latency endpoint hit once per step.

04 · Implementation

Two Ray Serve deployments, composed

The design is two Ray Serve deployments and a handle between them. WorldModel is the GPU tier: 32 replicas, one per H100, each holding the encoder and predictor. Planner is the driver: it runs the ten-round CEM loop, fans each round out across those replicas, and serves the endpoint the robot calls once per control step. Composing them this way, rather than as loose Ray Core actors, puts them under Serve's control: placement, health checks, restarts, and a declarative config a KubeRay RayService can roll out without downtime. The shape below elides the internals; what matters is where the fan-out sits.

serve_app.py — Ray Serve · the shape, details elided

```
from ray import serve
from ray.serve.handle import DeploymentHandle


# The GPU tier. One replica per H100, pinned, weights resident between calls.
@serve.deployment(
    num_replicas=32,
    ray_actor_options={"num_gpus": 1},
    max_ongoing_requests=1,          # one shard at a time, so fan-out maps 1:1
)
class WorldModel:
    def __init__(self, checkpoint): ...        # paid once, when Serve starts it
    def encode(self, frame, proprio): ...      # the encoder is part of the model
    def score(self, obs, goal, actions): ...   # one shard, one GPU batch


# The driver. Holds a handle to the GPU tier and runs the search.
@serve.deployment(num_replicas=2)
@serve.ingress(app)
class Planner:
    def __init__(self, model: DeploymentHandle):
        self.model = model

    @app.post("/plan")
    async def plan(self, frame, proprio, goal_image):
        obs, goal = await asyncio.gather(      # encode once per control step
            self.model.encode.remote(frame, proprio),
            self.model.encode.remote(goal_image),
        )
        mean, std = init_distribution(horizon, action_dim)

        for _ in range(10):                    # CEM rounds: sequential, the floor
            cands = sample(mean, std, n=800)
            scores = await asyncio.gather(*[   # fan out one shard per replica
                self.model.score.remote(obs, goal, shard)
                for shard in np.array_split(cands, 32)
            ])                                 # gather before the round refits
            mean, std = refit(cands, scores, elite_frac=0.1)
```

```
        return {"action": mean[0].tolist()}    # first action only, then re-plan


app_builder = Planner.bind(WorldModel.bind(CHECKPOINT))
```

Two choices carry the design. The GPU tier is pinned at num_replicas=32 rather than autoscaled, because a search whose replica count changes mid-round has unpredictable latency, and the point is a stable per-step budget. max_ongoing_requests=1 makes each replica take one shard at a time, so a 32-way fan-out lands one shard per GPU instead of queueing several on a subset. Underneath, each replica is still a Ray Core actor, which is why the weights stay resident; the difference is that Serve owns the lifecycle.

Source. The full implementation, including model loading, the complete CEM refinement step, and the KubeRay RayService manifest that provisions the four eight-GPU nodes, lives in the companion repository: [repository link to be added].

05 · Results

A modeled projection, not a benchmark, and why that distinction matters

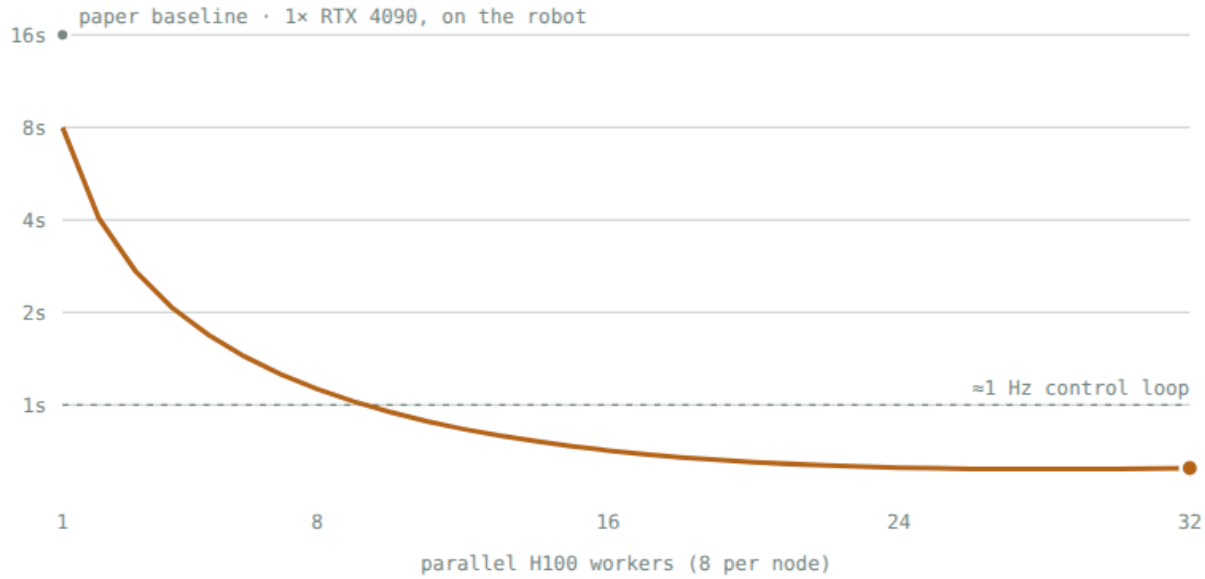
This is an engineering estimate derived from the paper's published numbers, not a measurement on a running cluster with a physical arm. It rests on one real anchor point, 16.0 seconds per action on a single RTX 4090, and a stated model of how that moves as rounds are distributed. Treat the curve as a sizing tool, not a wall-clock claim.

The model is simple. Each of the ten rounds costs a fixed compute time, 1.6 s on the paper's RTX 4090, calibrated so ten rounds reproduce 16.0 s exactly, plus a small coordination cost for dispatching shards and gathering results. Moving to an H100 halves the compute term, held deliberately conservative. Splitting a round across W workers divides what remains by W, with a light efficiency tax as W grows.

Planning latency vs. cluster size

H100 replicas, 8 per node.

0.62 s / action at 32 H100 workers (4 nodes) — ≈26× faster than the 16 s baseline



Model: time(W) = 10 × (0.8s / effective(W) + 0.001s × (W − 1)), where effective(W) = W / (1 + 0.008×(W−1)). The 0.8s per round is the paper's 1.6s on an RTX 4090 divided by a deliberately conservative 2× for an H100, which has roughly three times the 4090's memory bandwidth and dense tensor throughput. Constants are illustrative, chosen to reproduce the paper's baseline and to show plausible sublinear scaling, not measured from a deployed system.

What this projection does not capture. Real network latency between an on-premises robot and a cloud region, cold-start time the first time a replica handles a request, the actual H100-versus-4090 speedup for this specific model's operation mix, and any accuracy change from batching candidates differently than the paper did. All of these are measurable in an afternoon on a real KubeRay cluster; none of them are captured by a spreadsheet model, and this document does not claim otherwise.

06 · Further Optimization

What to try once the search itself is no longer the bottleneck

Once coordination overhead, not compute, is the thing standing between the system and sub-second planning, the next round of gains comes from restructuring the loop rather than adding GPUs.

A. Non-blocking control

Start planning step *t*+1 while the robot is still executing step *t*. That hides planning latency behind execution time instead of adding to it, and it is a controller change, not a model change.

B. Warm-started CEM

Receding-horizon control throws away everything but the first action. Re-using the rest to seed the next step's distribution, instead of a fresh Gaussian, lets each replan converge in fewer than

ten rounds, shrinking the sequential chain parallelism cannot touch.

C. Fleet-level batching

A 32-replica tier sized for one robot is idle between its requests. Behind a Serve endpoint the same tier serves a fleet, and request batching can pack shards from different robots into one GPU batch, amortizing 32 warm GPUs across many arms.

D. Model compression

Distilling or quantizing the predictor shrinks the compute term itself, independent of how many workers it is split across. It stacks with distribution rather than competing with it.

E. Serving-level optimization

The predictor rolls forward step by step over the horizon, so what LLM serving already relies on carries over: KV caching across rollout steps, prefill and decode disaggregation, continuous batching. These shrink the same compute term from a different direction, inside the same Serve deployment.

07 · Takeaways

The sixteen seconds is not an implementation defect. It follows from a deliberate architectural choice: a JEPA model predicts state rather than actions, so an action has to be searched for at inference time instead of read off a policy head. That is the same trade agentic systems make when they sample many candidates and score them, and it comes with the same benefit, a system that can be pointed at a new goal without retraining.

The four setups make the trade concrete. A hand-written policy buys millisecond actions and pays for them in task-specific engineering. V-JEPA2-AC trades that speed for a system that accepts any goal you can photograph. Moving the search to a cluster buys the speed back without giving up the generality, and does it in infrastructure rather than in the model. The projection puts four eight-GPU H100 nodes at roughly 0.6 seconds per action, with distribution rather than chip choice doing most of the work, and the ten sequential rounds setting a floor more hardware will not move. Attack that floor first, with non-blocking control and warm-started search.

References

1. Assran, M. et al. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning.* arXiv:2506.09985, 2025. arxiv.org/abs/2506.09985
2. Meta AI. *Introducing the V-JEPA 2 world model and new benchmarks for physical reasoning.* ai.meta.com/blog/v-jepa-2-world-model-benchmarks
3. Ray Project. *Ray Core: Key Concepts, Tasks and Actors.* docs.ray.io/en/latest/ray-core
4. Ray Project. *Ray Serve documentation.* docs.ray.io/en/latest/serve
5. Ray Project. *KubeRay: RayService.* docs.ray.io/en/latest/cluster/kubernetes

Planning-time and success-rate figures are as reported in the V-JEPA2-AC paper (arXiv:2506.09985). Cluster sizing, the CEM parallelization design, the latency model, and all

code shown are an independent engineering proposal built on top of that published work, not affiliated with or endorsed by Meta AI or the Ray project.